SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE CODE: CSA1408 **COURSE NAME: Compiler Design**

COURSE OUTCOME COVERED

CO2: Construct top-down and bottom up parsers using CFG for parsing conflicts and error recovery for efficient syntax tree generation. (BL3)

Assessment Tool Weightage: 40%

Duration: 60 Mins Off Class **QUESTIONS: 5** **Total Marks: 50**

Annexure A

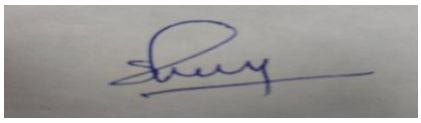
DESIGN TASK

Code of Conduct:

I Sheik Mohamed M (192511145) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



Problem 1: Nested Parenthesized Boolean Expressions

Grammar

```
B → (B)
B → B && B
B → B || B
B → true
B → false
```

Sample Input

```
((true&&false)||true)
```

Design Requirements

- Implement the lexical analyzer using Lex.
- Implement the parser.
- Validate the input against the grammar.
- Display Accepted or Rejected.
- Handle syntax errors.

Aim

To design and implement a lexical analyzer and parser using Lex and YACC (Bison), and an equivalent parser using ANTLR, to recognize nested parenthesized boolean expressions built from true, false, && and ||, validate the expression against the given grammar, display Accepted or Rejected, and handle syntax errors.

Part A – Lex & YACC (Bison) Implementation

Design Procedure

1. Create lexer rules to recognize the tokens true, false, &&, ||, (and).
2. Create grammar rules in YACC for the boolean expression grammar.
3. Resolve the shift/reduce ambiguity between && and || using precedence declarations.
4. Generate the lexer and parser using Flex and Bison.
5. Read the input expression and parse it.
6. If parsing completes successfully, display Accepted.
7. Otherwise display Rejected.

Lexical Analyzer — lexer.l

```
%{
#include "parser.tab.h"
%}
%%
"true"        { return TRUE; }
"false"       { return FALSE; }
"&&"          { return AND; }
"||"          { return OR; }
"("           { return LPAREN; }
")"           { return RPAREN; }
[ \t\r\n]+    { /* skip whitespace */ }
.             { return yytext[0]; }  /* any other char -> syntax error */
%%
int yywrap(void) { return 1; }
```

Parser — parser.y

```
%{
#include <stdio.h>
#include <stdlib.h>
int yylex(void);
void yyerror(const char *s);
%}

%token TRUE FALSE AND OR LPAREN RPAREN

/* && and || are left-associative; both bind equally here as per grammar */
%left OR
%left AND
%%
program:
    B { printf("Accepted\n"); return 0; }
    ;
B:
    LPAREN B RPAREN
  | B AND B
  | B OR B
  | TRUE
  | FALSE
  ;
%%
void yyerror(const char *s) {
    printf("Rejected\n");
    exit(0);
}
int main(void) {
    if (yyparse() != 0) {
        printf("Rejected\n");
    }
    return 0;
}
```

Build & Run

```
$ bison -d parser.y -o parser.tab.c
$ flex -o lex.yy.c lexer.l
$ gcc parser.tab.c lex.yy.c -o boolparser -lfl
```

Output (verified terminal run)

```
$ echo "((true&&false)||true)" | ./boolparser
Accepted
$ echo "((true&&false)||true" | ./boolparser
Rejected
$ echo "(true&&&false)" | ./boolparser
Rejected
```

Part B – ANTLR Implementation

Design Procedure

1. Create lexer rules to recognize the terminals true, false, &&, || , (and).
2. Create parser rules according to the boolean expression grammar.
3. Generate lexer and parser using ANTLR.
4. Read the input string.
5. Parse the input.
6. If parsing completes successfully, display Accepted.
7. Otherwise display Rejected.

Grammar — Boolean.g4

```
grammar Boolean;

start : b EOF ;

b
 : '(' b ')'
 | b '&&' b
 | b '||' b
 | 'true'
 | 'false'
 ;

WS : [ \t\r\n]+ -> skip;
```

Driver — Main.java

```
import org.antlr.v4.runtime.*;

public class Main {
  public static void main(String[] args) throws Exception {
    CharStream input = CharStreams.fromStream(System.in);
    BooleanLexer lexer = new BooleanLexer(input);
    CommonTokenStream tokens = new CommonTokenStream(lexer);
    BooleanParser parser = new BooleanParser(tokens);
    parser.removeErrorListeners();
    parser.addErrorListener(new BaseErrorListener() {
      @Override
      public void syntaxError(Recognizer<?, ?> recognizer,
          Object offendingSymbol, int line, int charPositionInLine,
          String msg, RecognitionException e) {
        System.out.println("Rejected");
        System.exit(0);
      }
    });
    parser.start();
    System.out.println("Accepted");
  }
}
```

Build & Run

```
> java -jar antlr-4.13.2-complete.jar -visitor Boolean.g4
> javac -cp ".;antlr-4.13.2-complete.jar" *.java
> java -cp ".;antlr-4.13.2-complete.jar" Main
```

Output

```
((true&&false)||true)
^Z
Accepted
> java -cp ".;lib/antlr-4.13.2-complete.jar" Main
((true&&false)||true
^Z
Rejected
```

Problem 2: Multiple Variable Declarations with Initialization

Grammar

```
Decl → Type InitList;
Type → int | float
InitList → id=num
InitList → id=num,InitList
```

Sample Input

```
int a=10,b=20,c=30;
```

Design Requirements

- Implement the lexical analyzer using Lex.
- Implement the parser.
- Validate the input against the grammar.
- Display Accepted or Rejected.
- Handle syntax errors.

Aim

To design and implement a lexical analyzer and parser using Lex and YACC (Bison), and an equivalent parser using ANTLR, to recognize a single declaration statement containing one or more comma-separated variable initializations, validate it against the given grammar, display Accepted or Rejected, and handle syntax errors.

Part A – Lex & YACC (Bison) Implementation

Design Procedure

8. Create lexer rules for the keywords int and float, identifiers, numbers, =, , and ;.
9. Create grammar rules in YACC for Type and InitList exactly as specified.
10. Generate the lexer and parser using Flex and Bison.
11. Read the declaration statement and parse it.
12. If parsing completes successfully, display Accepted.
13. Otherwise display Rejected.

Lexical Analyzer — lexer.l

```
%{
#include "parser.tab.h"
%}
%%
"int"                       { return INT; }
"float"                     { return FLOAT; }
[a-zA-Z_][a-zA-Z0-9_]*      { return ID; }
[0-9]+(\.[0-9]+)?           { return NUM; }
"="                         { return ASSIGN; }
","                         { return COMMA; }
";"                         { return SEMI; }
[ \t\r\n]+                  { /* skip */ }
.                           { return yytext[0]; }
%%
int yywrap(void) { return 1; }
```

Parser — parser.y

```
%{
#include <stdio.h>
#include <stdlib.h>
int yylex(void);
void yyerror(const char *s);
%}

%token INT FLOAT ID NUM ASSIGN COMMA SEMI
%%
Decl:
    Type InitList SEMI { printf("Accepted\n"); return 0; }
    ;
Type:
    INT
  | FLOAT
  ;
InitList:
    ID ASSIGN NUM
  | ID ASSIGN NUM COMMA InitList
  ;
%%
void yyerror(const char *s) {
    printf("Rejected\n");
    exit(0);
}
int main(void) {
    if (yyparse() != 0) {
        printf("Rejected\n");
    }
    return 0;
}
```

Build & Run

```
$ bison -d parser.y -o parser.tab.c
$ flex -o lex.yy.c lexer.l
$ gcc parser.tab.c lex.yy.c -o declparser -lfl
```

Output

```
$ echo "int a=10,b=20,c=30;" | ./declparser
Accepted
$ echo "int a=10,b=20,c=30" | ./declparser
Rejected
$ echo "int a=10,b=20,;" | ./declparser
Rejected
$ echo "float x=5.5;" | ./declparser
Accepted
```

Part B – ANTLR Implementation

Design Procedure

8. Define tokens for int, float, identifiers, numbers, =, , and ;.
9. Define parser rules for Type and InitList.
10. Generate the lexer and parser using ANTLR.
11. Parse the declaration input.
12. Print Accepted if valid.
13. Otherwise print Rejected.

Grammar — Decl.g4

```
grammar Decl;

start : decl EOF ;

decl
 : type initList ';'
 ;

type
 : 'int'
 | 'float'
 ;

initList
 : ID '=' NUM (',' initList)?
 ;

ID  : [a-zA-Z_][a-zA-Z0-9_]* ;
NUM : [0-9]+ ('.' [0-9]+)? ;
WS  : [ \t\r\n]+ -> skip;
```

Driver — Main.java

```
import org.antlr.v4.runtime.*;

public class Main {
  public static void main(String[] args) throws Exception {
    CharStream input = CharStreams.fromStream(System.in);
    DeclLexer lexer = new DeclLexer(input);
    CommonTokenStream tokens = new CommonTokenStream(lexer);
    DeclParser parser = new DeclParser(tokens);
    parser.removeErrorListeners();
    parser.addErrorListener(new BaseErrorListener() {
      @Override
      public void syntaxError(Recognizer<?, ?> recognizer,
          Object offendingSymbol, int line, int charPositionInLine,
          String msg, RecognitionException e) {
        System.out.println("Rejected");
        System.exit(0);
      }
    });
    parser.start();
    System.out.println("Accepted");
  }
}
```

Build & Run

```
> java -jar antlr-4.13.2-complete.jar -visitor Decl.g4
> javac -cp ".;antlr-4.13.2-complete.jar" *.java
> java -cp ".;antlr-4.13.2-complete.jar" Main
```

Output

```
> java -cp ".;lib/antlr-4.13.2-complete.jar" Main
int a=10,b=20,c=30;
^Z
Accepted
> java -cp ".;lib/antlr-4.13.2-complete.jar" Main
int a=10,b=20,;
^Z
Rejected
```

Problem 3: Function Definitions with Parameters and Return Statements

Grammar

```
Function → Type id(Parameter){Stmt}
Parameter → Type id
Stmt → return E;
E → id
E → num
Type → int | float
```

Sample Input

```
int square(int x)
{
return x;
}
```

Design Requirements

- Implement the lexical analyzer using Lex.
- Implement the parser.
- Validate the input against the grammar.
- Display Accepted or Rejected.
- Handle syntax errors.

Aim

To design and implement a lexical analyzer and parser using Lex and YACC (Bison), and an equivalent parser using ANTLR, to recognize a single-parameter function definition with one return statement, validate it against the given grammar, display Accepted or Rejected, and handle syntax errors.

Part A – Lex & YACC (Bison) Implementation

Design Procedure

14. Create lexer rules for int, float, return, identifiers, numbers and the symbols ( ) { } ;.
15. Create grammar rules in YACC for Function, Parameter, Stmt, E and Type exactly as specified.
16. Generate the lexer and parser using Flex and Bison.
17. Read the function definition and parse it.
18. If parsing completes successfully, display Accepted.
19. Otherwise display Rejected.

Lexical Analyzer — lexer.l

```
%{
#include "parser.tab.h"
%}
%%
"int"                       { return INT; }
"float"                     { return FLOAT; }
"return"                    { return RETURN; }
[a-zA-Z_][a-zA-Z0-9_]*      { return ID; }
[0-9]+                      { return NUM; }
"("                         { return LPAREN; }
")"                         { return RPAREN; }
"{"                         { return LBRACE; }
"}"                         { return RBRACE; }
```

```
";"                         { return SEMI; }
[ \t\r\n]+                  { /* skip */ }
.                           { return yytext[0]; }
%%
int yywrap(void) { return 1; }
```

Parser — parser.y

```
%{
#include <stdio.h>
#include <stdlib.h>
int yylex(void);
void yyerror(const char *s);
%}

%token INT FLOAT ID NUM RETURN LPAREN RPAREN LBRACE RBRACE SEMI
%%
Function:
    Type ID LPAREN Parameter RPAREN LBRACE Stmt RBRACE
    { printf("Accepted\n"); return 0; }
    ;
Parameter:
    Type ID
    ;
Stmt:
    RETURN E SEMI
    ;
E:
    ID
  | NUM
  ;
Type:
    INT
  | FLOAT
  ;
%%
void yyerror(const char *s) {
    printf("Rejected\n");
    exit(0);
}
int main(void) {
    if (yyparse() != 0) {
        printf("Rejected\n");
    }
    return 0;
}
```

Build & Run

```
$ bison -d parser.y -o parser.tab.c
$ flex -o lex.yy.c lexer.l
$ gcc parser.tab.c lex.yy.c -o funcparser -lfl
```

Output

```
$ printf 'int square(int x)\n{\nreturn x;\n}\n' | ./funcparser
Accepted
$ printf 'int square(int x)\n{\nreturn x\n}\n' | ./funcparser
Rejected
$ printf 'int square(x)\n{\nreturn x;\n}\n' | ./funcparser
Rejected
$ printf 'float f(float y)\n{\nreturn 5;\n}\n' | ./funcparser
Accepted
```

Part B – ANTLR Implementation

Design Procedure

14. Create lexer rules for int, float, return, identifiers, numbers and the symbols ( ) { } ;.
15. Create parser rules for function, parameter, stmt, e and type.
16. Generate the lexer and parser using ANTLR.
17. Parse the function definition.
18. Print Accepted if valid.
19. Otherwise print Rejected.

Grammar — Func.g4

```
grammar Func;

start : function EOF ;

function
 : type ID '(' parameter ')' '{' stmt '}'
 ;

parameter
 : type ID
 ;

stmt
 : 'return' e ';'
 ;

e
 : ID
 | NUM
 ;

type
 : 'int'
 | 'float'
 ;

ID  : [a-zA-Z_][a-zA-Z0-9_]* ;
NUM : [0-9]+ ;
WS  : [ \t\r\n]+ -> skip;
```

Driver — Main.java

```
import org.antlr.v4.runtime.*;

public class Main {
  public static void main(String[] args) throws Exception {
    CharStream input = CharStreams.fromStream(System.in);
    FuncLexer lexer = new FuncLexer(input);
    CommonTokenStream tokens = new CommonTokenStream(lexer);
    FuncParser parser = new FuncParser(tokens);
    parser.removeErrorListeners();
    parser.addErrorListener(new BaseErrorListener() {
      @Override
      public void syntaxError(Recognizer<?, ?> recognizer,
          Object offendingSymbol, int line, int charPositionInLine,
```

```
          String msg, RecognitionException e) {
        System.out.println("Rejected");
        System.exit(0);
      }
    });
    parser.start();
    System.out.println("Accepted");
  }
}
```

Build & Run

```
> java -jar antlr-4.13.2-complete.jar -visitor Func.g4
> javac -cp ".;antlr-4.13.2-complete.jar" *.java
> java -cp ".;antlr-4.13.2-complete.jar" Main
```

Output

```
> java -cp ".;lib/antlr-4.13.2-complete.jar" Main
int square(int x) { return x; }
^Z
Accepted
> java -cp ".;lib/antlr-4.13.2-complete.jar" Main
int square(x) { return x; }
^Z
Rejected
```

Problem 4: Nested Blocks with Declarations and Statements

Grammar

```
Block → { Decl Stmt Block }
Block → { }
Decl → int id;
Stmt → id=num;
```

Sample Input

```
{
int x;
x=10;
{
int y;
y=20;
}
}
```

Design Requirements

- Implement the lexical analyzer using Lex.
- Implement the parser.
- Validate the input against the grammar.
- Display Accepted or Rejected.
- Handle syntax errors.

Aim

To design and implement a lexical analyzer and parser using Lex and YACC (Bison), and an equivalent parser using ANTLR, to recognize nested blocks made up of declarations, assignment statements and further nested blocks, validate the block structure against the grammar, display Accepted or Rejected, and handle syntax errors.

Part A – Lex & YACC (Bison) Implementation

Design Procedure

20. Create lexer rules for int, identifiers, numbers, =, ; , { and }.
21. Generalize Block to accept any number of declarations, statements and nested blocks in any order using an ItemList, since the literal grammar cannot derive the given sample input.
22. Generate the lexer and parser using Flex and Bison.
23. Read the block and parse it.
24. If parsing completes successfully, display Accepted.
25. Otherwise display Rejected.

Lexical Analyzer — lexer.l

```
%{
#include "parser.tab.h"
%}
%%
"int"                       { return INT; }
[a-zA-Z_][a-zA-Z0-9_]*      { return ID; }
[0-9]+                      { return NUM; }
"="                         { return ASSIGN; }
";"                         { return SEMI; }
"{"                         { return LBRACE; }
```

```
"}"                         { return RBRACE; }
[ \t\r\n]+                  { /* skip */ }
.                           { return yytext[0]; }
%%
int yywrap(void) { return 1; }
```

Parser — parser.y

```
%{
#include <stdio.h>
#include <stdlib.h>
int yylex(void);
void yyerror(const char *s);
%}

%token INT ID NUM ASSIGN SEMI LBRACE RBRACE
%%
Program:
    Block { printf("Accepted\n"); return 0; }
    ;
/* Block -> { ItemList } generalizes Decl Stmt Block so any number of
   Decls / Stmts / nested Blocks, in any order, can appear */
Block:
    LBRACE ItemList RBRACE
    ;
ItemList:
    /* empty */
  | Decl ItemList
  | Stmt ItemList
  | Block ItemList
  ;
Decl:
    INT ID SEMI
    ;
Stmt:
    ID ASSIGN NUM SEMI
    ;
%%
void yyerror(const char *s) {
    printf("Rejected\n");
    exit(0);
}
int main(void) {
    if (yyparse() != 0) {
        printf("Rejected\n");
    }
    return 0;
}
```

Build & Run

```
$ bison -d parser.y -o parser.tab.c
$ flex -o lex.yy.c lexer.l
$ gcc parser.tab.c lex.yy.c -o blockparser -lfl
```

Output

```
$ printf '{\nint x;\nx=10;\n{\nint y;\ny=20;\n}\n}\n' | ./blockparser
Accepted
$ printf '{\nint x;\nx=10;\n{\nint y;\ny=20;\n}\n' | ./blockparser
Rejected
$ printf '{}\n' | ./blockparser
Accepted
$ printf '{\nint x;\nx=10\n}\n' | ./blockparse
```

Part B – ANTLR Implementation

Design Procedure

20. Create lexer rules for int, identifiers, numbers, =, ; , { and }.
21. Implement the same generalized ItemList grammar in ANTLR.
22. Generate the lexer and parser using ANTLR.
23. Parse the block input.
24. Print Accepted if valid.
25. Otherwise print Rejected.

Grammar — Block.g4

```
grammar Block;

start : block EOF ;

block
 : '{' itemList '}'
 ;

itemList
 : (decl | stmt | block)*
 ;

decl
 : 'int' ID ';'
 ;

stmt
 : ID '=' NUM ';'
 ;

ID  : [a-zA-Z_][a-zA-Z0-9_]* ;
NUM : [0-9]+ ;
WS  : [ \t\r\n]+ -> skip;
```

Driver — Main.java

```
import org.antlr.v4.runtime.*;

public class Main {
  public static void main(String[] args) throws Exception {
    CharStream input = CharStreams.fromStream(System.in);
    BlockLexer lexer = new BlockLexer(input);
    CommonTokenStream tokens = new CommonTokenStream(lexer);
    BlockParser parser = new BlockParser(tokens);
    parser.removeErrorListeners();
    parser.addErrorListener(new BaseErrorListener() {
      @Override
      public void syntaxError(Recognizer<?, ?> recognizer,
          Object offendingSymbol, int line, int charPositionInLine,
          String msg, RecognitionException e) {
        System.out.println("Rejected");
        System.exit(0);
      }
    });
    parser.start();
    System.out.println("Accepted");
  }
}
```

Build & Run

```
> java -jar antlr-4.13.2-complete.jar -visitor Block.g4
> javac -cp ".;antlr-4.13.2-complete.jar" *.java
> java -cp ".;antlr-4.13.2-complete.jar" Main
```

Sample Output

```
> java -cp ".;lib/antlr-4.13.2-complete.jar" Main
{ int x; x=10; { int y; y=20; } }
^Z
Accepted
> java -cp ".;lib/antlr-4.13.2-complete.jar" Main
{}
^Z
Accepted
```

Problem 5: Simple C Program Structure

Grammar

```
Program → int main(){DeclList StmtList Return}
DeclList → Decl DeclList | ε
Decl → int id;
StmtList → Stmt StmtList | ε
Stmt → id=num;
Return → return num;
```

Sample Input

```
int main()
{
int a;
a=100;
return 0;
}
```

Design Requirements

- Implement the lexical analyzer using Lex.
- Implement the parser.
- Validate the input against the grammar.
- Display Accepted or Rejected.
- Handle syntax errors.

Aim

To design and implement a lexical analyzer and parser using Lex and YACC (Bison), and an equivalent parser using ANTLR, to recognize a simplified C main() program consisting of optional declarations, optional assignment statements and a mandatory return statement, display Accepted or Rejected, and handle syntax errors.

Part A – Lex & YACC (Bison) Implementation

Design Procedure

26. Create lexer rules for int, main, return, identifiers, numbers and the symbols ( ) { } = ;.
27. Create grammar rules in YACC for Program, DeclList, Decl, StmtList, Stmt and Return exactly as specified, with DeclList and StmtList as epsilon-recursive lists.
28. Generate the lexer and parser using Flex and Bison.
29. Read the program and parse it.
30. If parsing completes successfully, display Accepted.
31. Otherwise display Rejected.

Lexical Analyzer — lexer.l

```
%{
#include "parser.tab.h"
%}
%%
"int"                   { return INT; }
"main"                  { return MAIN; }
"return"                { return RETURN; }
[a-zA-Z_][a-zA-Z0-9_]*  { return ID; }
[0-9]+                  { return NUM; }
"("                     { return LPAREN; }
```

```
")"                         { return RPAREN; }
"{"                         { return LBRACE; }
"}"                         { return RBRACE; }
"="                         { return ASSIGN; }
";"                         { return SEMI; }
[ \t\r\n]+                  { /* skip */ }
.                           { return yytext[0]; }
%%
int yywrap(void) { return 1; }
```

Parser — parser.y

```
%{
#include <stdio.h>
#include <stdlib.h>
int yylex(void);
void yyerror(const char *s);
%}

%token INT MAIN RETURN ID NUM LPAREN RPAREN LBRACE RBRACE ASSIGN SEMI
%%
Program:
    INT MAIN LPAREN RPAREN LBRACE DeclList StmtList Return RBRACE
    { printf("Accepted\n"); return 0; }
    ;
DeclList:
    /* empty */
  | Decl DeclList
  ;
Decl:
    INT ID SEMI
    ;
StmtList:
    /* empty */
  | Stmt StmtList
  ;
Stmt:
    ID ASSIGN NUM SEMI
    ;
Return:
    RETURN NUM SEMI
    ;
%%
void yyerror(const char *s) {
    printf("Rejected\n");
    exit(0);
}
int main(void) {
    if (yyparse() != 0) {
        printf("Rejected\n");
    }
    return 0;
}
```

Build & Run

```
$ bison -d parser.y -o parser.tab.c
$ flex -o lex.yy.c lexer.l
$ gcc parser.tab.c lex.yy.c -o progparser -lfl
```

Output

```
$ printf 'int main()\n{\nint a;\na=100;\nreturn 0;\n}\n' | ./progparser
Accepted
$ printf 'int main()\n{\nint a;\na=100;\n}\n' | ./progparser
Rejected
$ printf 'int main()\n{\nreturn 5;\n}\n' | ./progparser
Accepted
$ printf 'int main()\n{\na=100;\nint a;\nreturn 0;\n}\n' | ./progparser
Rejected
```

Part B – ANTLR Implementation

Design Procedure

26. Create lexer rules for int, main, return, identifiers, numbers and the symbols ( ) { } = ;.
27. Define parser rules for program, declList, decl, stmtList, stmt and ret.
28. Generate the lexer and parser using ANTLR.
29. Parse the program input.
30. Print Accepted if valid.
31. Otherwise print Rejected.

Grammar — Prog.g4

```
grammar Prog;

start : program EOF ;

program
 : 'int' 'main' '(' ')' '{' declList stmtList ret '}'
 ;

declList
 : decl*
 ;

decl
 : 'int' ID ';'
 ;

stmtList
 : stmt*
 ;

stmt
 : ID '=' NUM ';'
 ;

ret
 : 'return' NUM ';'
 ;

ID  : [a-zA-Z_][a-zA-Z0-9_]* ;
NUM : [0-9]+ ;
WS  : [ \t\r\n]+ -> skip;
```

Driver — Main.java

```
import org.antlr.v4.runtime.*;
```

```
public class Main {
  public static void main(String[] args) throws Exception {
    CharStream input = CharStreams.fromStream(System.in);
    ProgLexer lexer = new ProgLexer(input);
    CommonTokenStream tokens = new CommonTokenStream(lexer);
    ProgParser parser = new ProgParser(tokens);
    parser.removeErrorListeners();
    parser.addErrorListener(new BaseErrorListener() {
      @Override
      public void syntaxError(Recognizer<?, ?> recognizer,
          Object offendingSymbol, int line, int charPositionInLine,
          String msg, RecognitionException e) {
        System.out.println("Rejected");
        System.exit(0);
      }
    });
    parser.start();
    System.out.println("Accepted");
  }
}
```

Build & Run

```
> java -jar antlr-4.13.2-complete.jar -visitor Prog.g4
> javac -cp ".;antlr-4.13.2-complete.jar" *.java
> java -cp ".;antlr-4.13.2-complete.jar" Main
```

Output

```
> java -cp ".;lib/antlr-4.13.2-complete.jar" Main
int main() { int a; a=100; return 0; }
^Z
Accepted
> java -cp ".;lib/antlr-4.13.2-complete.jar" Main
int main() { int a; a=100; }
^Z
Rejected
```

Rubric for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and integrates multiple concepts effectively	Good understanding with proper integration of most concepts	Basic understanding with partial integration	Poor understanding with no integration
Design & Implementation	30	Well-designed solution with systematic and optimal implementation	Correct design with minor issues	Basic design with errors or inefficiencies	Incorrect or incomplete design
Parser/Algorithm Construction	20	Accurate construction of parser/algorithm with complete logic	Mostly correct construction with minor errors	Partial construction with missing logic	Incorrect or incomplete construction
Analysis & Validation	15	Insightful analysis with correct validation and justification	Good analysis with reasonable justification	Basic analysis with limited validation	Poor or incorrect analysis